# Towards a positive version of Cook's PV

*Workshop on Proof Complexity 2026*

---

Adriana Baldacchino, Anupam Das

19th July 2026

University of Birmingham

# Table of contents

1. Motivation

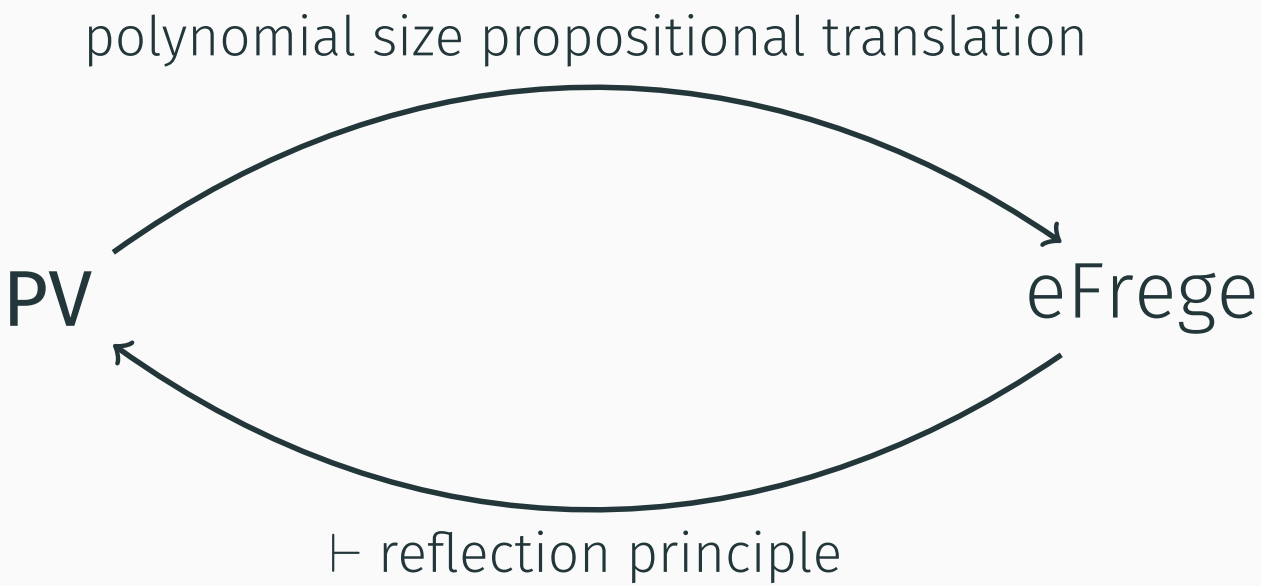
2. Positive polynomial time functions

3. The system **posPV**

4. Conclusion

# Motivation

# Cook's correspondence




Can we establish a *positive* version of this correspondence?

# Why study these functions?

- Positive functions are well studied in *monotone circuit complexity*, in particular due to the seminal results of Razborov separating **P** from **NP** in this setting [14, 1].
- These results inspired investigation of the *monotone sequent calculus* MLK [13, 4]. Surprisingly, it was found that MLK polynomially simulates LK over positive formulas [2, 3].
- Could we bring these results into the uniform setting, by extending work on uniform monotone complexity [9, 11, 12, 7]?

# Positive polynomial time functions

# Our setting

We will mainly work with binary words $w \in \{0,1\}^*$, generated by $\epsilon, s_0, s_1$, where $\epsilon$ is the empty string and

$$s_0 x = x0$$

$$s_1 x = x1$$

We study functions on these words, $f, g : \{0,1\}^* \to \{0,1\}^*$.

# Positive polynomial-time functions

We study the *positive polynomial time functions* **posFP** which are functions on $\{0,1\}^*$ computable by $\Delta_0$-uniform families of $\neg$-free circuits.

**Example**

The threshold function is a positive polynomial time function:

$$\begin{aligned} \mathrm{Th}(x,0) &:= 1 \\ \mathrm{Th}(\epsilon, n+1) &:= 0 \\ \mathrm{Th}(s_i(x), n+1) &:= (i \wedge \mathrm{Th}(x,n)) \vee (\neg i \wedge \mathrm{Th}(x, n+1)) \end{aligned}$$

# Function algebra characterisation

In order to establish a positive theory of **PV**, we use the function algebra characterisation of **posFP** [7]. This is the smallest class containing some initial functions and closed under composition and *uniform bounded recursion*, in which $f$ is defined from $g, h$ as

$$\begin{aligned} f(\epsilon, \vec{x}) &= g(\vec{x}) \\ f(s_i x, \vec{x}) &= h(i, x, \vec{x}, f(x, \vec{x})) \end{aligned}$$

provided that $f(x, \vec{x}) \leq b(x, \vec{x})$ for some $b \in \mathbf{posFP}$.

This version of recursion does not allow us to recover conditionals on bits. To regain some of the functions we lose because of this, we add as initial functions the meet and join functions $\wedge, \vee$.

# Monotonicity

We define the order $\leq$ on binary words to be

$$w \leq v \quad \text{if and only if} \quad |w| = |v| \text{ and } w(i) \leq v(i) \text{ for } 0 \leq i < |w|$$

**Lemma (Monotonicity)**

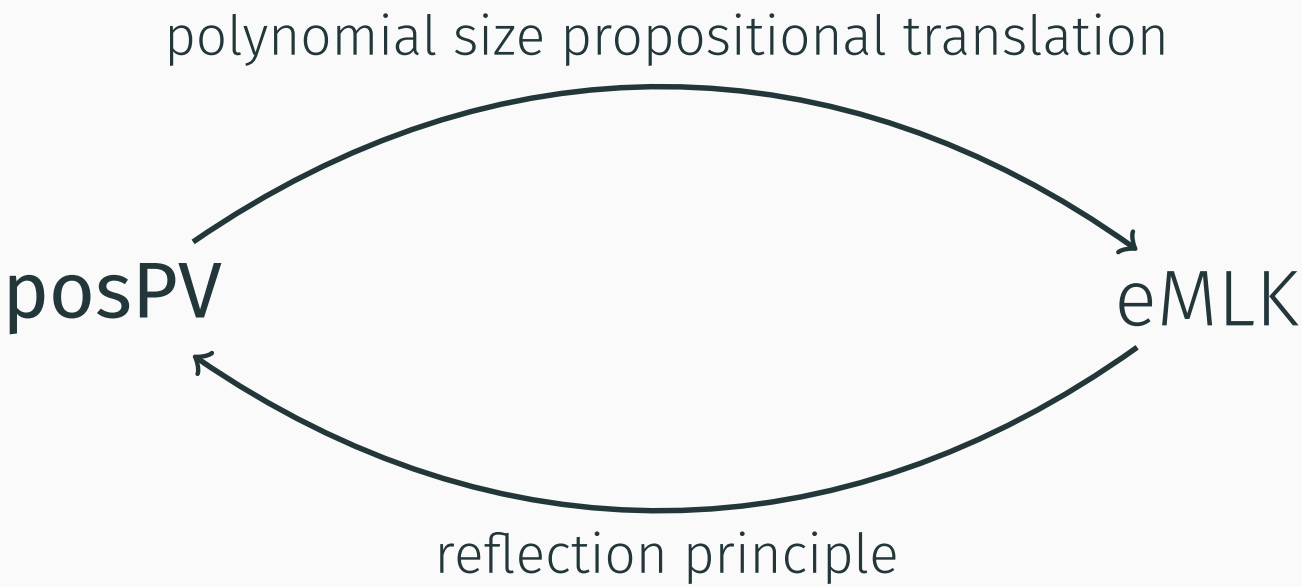
*For $f \in$ posFP, $x \leq y$ implies that $f(x) \leq f(y)$.*

This implies that the size of the input of a function determines the size of the output of the function. In fact, for every $f \in$ **posFP**, we can effectively find $len_f : \mathbb{N} \to \mathbb{N}$ with $len_f \in \mathcal{E}^2 =$ **FLINSPACE** such that $len_f(|\vec{x}|) = |f(\vec{x})|$ [7].

Razborov's result implies that there exist monotone functions in **FP** which do not belong in **posFP** [15].

# The system posPV

# A monotone version of Cook's correspondence



# Translating a term into eMLK formulas

For a fixed input size $\vec{l}$, a term (corresponding to a function $f \in \mathbf{posFP}$) is translated into multiple eMLK formulas.

$$f(\vec{x}) \quad \rightsquigarrow \quad \langle f \rangle^{\vec{l}}$$

Here we use the fact that the length of the input determines the length of the output: $\langle f \rangle^{\vec{l}}$ is a tuple of length $len_f(\vec{l})$ where the $i$th entry is an eMLK formula representing the $i$th output bit.

$$s_1(x) \quad \rightsquigarrow \quad \langle (x(0), x(1), \ldots, x(l-1), \top) \rangle^{l}$$

# Our (in)equational theory posPV

The monotone sequent calculus eMLK gives us judgements of the form $A \Rightarrow B$, which means that $A$ implies $B$. Naturally, this leads to the correspondence

$$f(\vec{x}) \leq g(\vec{x}) \quad \rightsquigarrow \quad \text{for all } \vec{l},\ len_f(\vec{l}) = len_g(\vec{l}) \text{ and } \langle f \rangle^{\vec{l}}(i) \Rightarrow \langle g \rangle^{\vec{l}}(i),\ 0 \leq i < len_f(\vec{l}).$$

Because of this, it makes sense to build our theory **posPV** as an *inequational* theory, with judgements of the form $f(\vec{x}) \leq g(\vec{x})$ as the basic atoms of our system.

# Axiomatising posPV

Our system of **posPV** is axiomatised similarly to Cook's theory, with various basic axioms such as reflexivity of $\epsilon$, $\vdash \epsilon \leq \epsilon$, and behaviour of generating functions $x \leq y \vdash s_0 x \leq s_0 y$.

In particular for each defining equation $s = t$ for a **posFP** function $f$, we add both $\vdash s \leq t$ and $\vdash t \leq s$ as axioms.

# The induction rule

We are able to express induction only on a single inequality:

$$\textit{ind}\ \frac{\Gamma \vdash A(\epsilon) \quad \Gamma, A(x) \vdash A(s_0 x) \quad \Gamma, A(x) \vdash A(s_1 x)}{\Gamma \vdash A(y)}\ x \text{ not in } \Gamma$$

This allows us to prove for example, general reflection

$$\textit{ind}\ \frac{\textit{refl}\ \dfrac{}{\vdash \epsilon \leq \epsilon} \qquad \textit{s-intro}\ \dfrac{}{x \leq x \vdash s_0 x \leq s_0 x} \qquad \textit{s-intro}\ \dfrac{}{x \leq x \vdash s_1 x \leq s_1 x}}{\vdash y \leq y}$$

# Limitations of induction

Usually, a rule like this is enough to also express induction on implications, as one would express $A \vdash B$ as $A \to B$. However, since implications are not monotone, we cannot do this, and in fact induction is less powerful in this system.

For example, true statements we are unable to prove include

$$x \leq \epsilon \vdash \epsilon \leq x,$$

which needs a case analysis on $x$, and even monotonicity:

$$x \leq y \vdash f(x) \leq f(y).$$

# Monotonicity and cases

To regain expressivity, we add the *cases* rule

$$\textit{cases}\ \frac{\Gamma(\epsilon) \vdash A(\epsilon) \quad \Gamma(s_0 x) \vdash A(s_0 x) \quad \Gamma(s_1 x) \vdash A(s_1 x)}{\Gamma(y) \vdash A(y)}$$

and the monotonicity axiom $x \leq y \vdash t(x, \vec{z}) \leq t(y, \vec{z})$.

Note that we can't write the cases rule as an axiom, as the way one would typically do this is using implications.

# Difficulties of coding in posPV

For this system, we prove both directions of the correspondence. The main difficulties lie in coding a proof into **posPV**. Because of the lack of negations, no known efficient *pairing functions* can be coded in *m***PV**, which means there is no straightforward way to code arbitrary length lists.

Another stumbling point is the lack of conditionals on bits. We manage to recover some conditional reasoning from the fact that we have *conditionals on lengths*.

Some of our results in this direction include:

- Coding the functions of $\mathcal{E}^2$ as unary functions in **posPV**;
- Coding conjunctions;
- Coding bounded universal quantification over unary words.

# Conclusion

# Conclusions and further directions

- We have established a system for positive **PV** and proved that **posPV** has polynomial size translations into eMLK, and that **posPV** proves the reflection principle for eMLK.
- It would be interesting to study the strength of **posPV** further, by investigating possible witness theorems.
- This framework could also be used to adapt the proof that eMLK polynomially simulates eLK $\restriction_{pos}$ to the uniform setting.

Thank you! Questions?

# References i



N. Alon and R. B. Boppana.
**The monotone circuit complexity of boolean functions.**
*Combinatorica*, 7(1):1–22, 1987.


A. Atserias, N. Galesi, and P. Pudlák.
**Monotone simulations of non-monotone proofs.**
*Journal of Computer and System Sciences*, 65(4):626–638, 2002.
Special Issue on Complexity 2001.


S. Buss, V. Kabanets, A. Kolokolova, and M. Koucký.
**Expander construction in VNC1.**
*Annals of Pure and Applied Logic*, 171(7):102796, 2020.

# References ii




M. Bílková.

**Monotone sequent calculus and resolution.**

*Commentationes Mathematicae Universitatis Carolinae*, 42(3):575–582, 2001.



A. Cobham.

**The intrinsic computational difficulty of functions.**

In Y. Bar-Hillel, editor, *Logic, methodology and philosophy of science*, pages 24–30. North-Holland Pub. Co., 1965.



S. A. Cook.

**Feasibly constructive proofs and the propositional calculus (preliminary version).**

In *Proceedings of the Seventh Annual ACM Symposium on Theory of Computing*, STOC '75, page 83–97, New York, NY, USA, 1975. Association for Computing Machinery.

# References iii




A. Das and I. Oitavem.
**A Recursion-Theoretic Characterisation of the Positive Polynomial-Time Functions.**
In D. R. Ghica and A. Jung, editors, *27th EACSL Annual Conference on Computer Science Logic (CSL 2018)*, volume 119 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:17, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.



M. Grigni.
***Structure in monotone complexity.***
PhD thesis, USA, 1991.
AAI0571417.

# References iv



M. Grigni and M. Sipser.
**Monotone complexity.**
In *Proceedings of the London Mathematical Society Symposium on Boolean Function Complexity*, page 57–75, USA, 1992. Cambridge University Press.


J. Krajíček.
***Proof Complexity.***
Cambridge University Press, 2019.


C. Lautemann, T. Schwentick, and I. A. Stewart.
**On positive P.**
In *Proceedings of the 11th Annual IEEE Conference on Computational Complexity*, CCC '96, page 162, USA, 1996. IEEE Computer Society.

# References v




C. Lautemann, T. Schwentick, and I. A. Stewart.
**Positive versions of polynomial time.**
*Information and Computation*, 147(2):145–170, 1998.



P. Pudlák and S. R. Buss.
**How to lie without being (easily) convicted and the lengths of proofs in propositional calculus.**
In L. Pacholski and J. Tiuryn, editors, *Computer Science Logic*, pages 151–162, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.



A. Razborov.
**Lower bounds on the monotone complexity of some boolean function.**
In *Soviet Math. Dokl.*, volume 31, pages 354–357, 1985.

# References vi


A. A. Razborov.
**Lower bounds on monotone complexity of the logical permanent.**
*Mathematical notes of the Academy of Sciences of the USSR*, 37(6):485–493, 1985.